

GHOST-HUNTER / HIDASS: A Host Intrusion Detection & Active Signaling System via eBPF-Instrumented Bayesian Jitter Injection and Shannon-Entropy-Driven Behavioral Profiling of Covert Threat Actors

Taha Kouiyasse

Abstract—Contemporary Host Intrusion Detection Systems (HIDS) rely on threshold-based kill-switch semantics: upon anomaly confirmation the offending session is terminated and the operator is alerted. Against a sophisticated, manually-operated adversary—foreign intelligence operatives, military-grade Advanced Persistent Threat (APT) groups, or deep-cover actors executing interactive commands via pseudo-TTY channels—this approach is catastrophically self-defeating. The moment the connection drops, the adversary knows they have been discovered; they execute anti-forensic routines, purge logs, and pivot to dormant backdoors before any meaningful intelligence can be collected. GHOST-HUNTER implements the antithetical paradigm: instead of ejecting the threat, it *deceives, degrades, and diagnoses* it. The system attaches zero-copy eBPF probes at the `tty_write` and `tty_read` kernel tracepoints, computes inter-keystroke timing intervals Δt_k at nanosecond resolution, and feeds a continuous Shannon entropy evaluator on sliding telemetry windows to discriminate human typing cadence from automated tool execution or pasted exploit strings. The session interaction is modelled as a Sequential Imperfect-Information Stackelberg Signaling Game in which the defender holds a private information advantage. A Bayesian dispatcher—operating exclusively in fixed-point integer arithmetic within the `no_std` kernel scheduler to preclude floating-point exceptions—dynamically adjusts the scale parameter λ of an exponential jitter distribution injected into TTY write descriptors, inducing measurable latency imperceptible to casual users but operationally crippling to scripted attack suites. The system is implemented as a four-crate Rust workspace: `gh-probe` (kernel-space, `no_std`, zero heap allocation), `gh-analyzer` (user-space, async Tokio runtime, four worker threads), `gh-common` (ABI-stable shared types), and `xtask` (build orchestration). Clippy verification and live execution traces confirm structural correctness and graceful asynchronous shutdown with full kernel-state restoration.

Index Terms—eBPF, Host Intrusion Detection, Deception Technology, Behavioral Biometrics, Shannon Entropy, Bayesian Inference, Stackelberg Game Theory, Rust `no_std`, Zero-Allocation Kernel Programming, TTY Jitter Injection, APT Attribution, Active Cyber Defense.



1 INTRODUCTION

The discipline of host-based intrusion detection has, for three decades, operated under a tacit assumption: detection and response are sequential, mutually exclusive operations. A sensor observes; it fires an alert; an operator—or an automated runbook—terminates the session and patches the vector. This pipeline is adequate against opportunistic threat actors executing scripted, non-interactive payloads. It is wholly inadequate against the class of adversary that GHOST-HUNTER is designed to confront.

The manually-operated, intelligence-trained threat actor—hereafter denoted the *covert operator*—is a qualitatively different adversary. They adapt in real time, interpret subtle environmental cues, maintain operational security (OPSEC) discipline, and operate under the assumption that detection is always possible. When a connection drops unexpectedly, the sophisticated operator does not restart their session; they execute a well-rehearsed anti-forensic contingency: log truncation via `/proc/$pid/fd` manipulation, history

suppression through `HISTFILE=/dev/null`, backdoor activation across a pre-positioned C2 channel, and exfiltration of any staged intelligence before the window closes. The defender’s hasty termination thus not only destroys the opportunity for intelligence collection, it actively accelerates the adversary’s cleanup operations.

GHOST-HUNTER inverts this logic. Derived from the deception doctrine of *active cyber defense* and the intelligence discipline of *controlled elicitation*, the protocol’s core operational objective is threefold:

- 1) **Decelerate:** inject calibrated, chaotic latency into the adversary’s TTY channel, degrading their operational tempo without triggering their OPSEC tripwires.
- 2) **Profile:** harvest a high-fidelity behavioral signature—inter-keystroke timing, command entropy, tool execution patterns—that constitutes a persistent attribution artifact.
- 3) **Deceive:** maintain the adversary’s belief that the session is unmonitored, inducing them to reveal their full operational playbook, exfiltration targets, and

• Lead System Architect, AI-Augmented eBPF & Rust Engineering, Sovereign Systems Research. E-mail: tahakouiyasse@protonmail.com

lateral-movement vectors.

The remainder of this paper is structured as follows. Section 2 articulates the AI-Augmented Systems Design methodology that governs the project. Section 3 formalizes the threat model. Section 4 develops the complete mathematical framework. Section 5 establishes the Stackelberg game-theoretic model. Section 6 details the four-crate Rust workspace architecture. Section 7 presents critical implementation details including fixed-point arithmetic and eBPF probe design. Section 8 provides verified console execution traces. Section 9 discusses detection efficacy and operational security properties. Section 10 surveys related work. Section 11 presents conclusions and directions for future development.

2 AI-AUGMENTED SYSTEMS DESIGN PARADIGM

2.1 A New Engineering Philosophy

The architecture of GHOST-HUNTER was conceived and directed under an engineering paradigm that warrants explicit articulation, as it represents a meaningful departure from the conventional model of systems security research. In the traditional academic pipeline, a researcher simultaneously occupies the roles of theoretician, architect, and implementer. The depth of contribution is bounded by the individual's capacity to traverse the full vertical stack: from information-theoretic formalism at the top, through algorithm design, through systems architecture, to compiler-level syntax and ABI constraints at the bottom.

The Lead System Architect of this project, Taha Kouiyasse, operates under a different epistemological commitment. The primary cognitive contribution is *systemic*: the identification and formalization of the absolute operational boundaries of the problem—the physics of kernel scheduling, the information geometry of timing side-channels, the strategic structure of adversarial interaction, the topological constraints of lock-free ring-buffer data flows. These concerns exist at a level of abstraction at which compiler syntax is irrelevant and at which no amount of Rust ownership familiarity substitutes for genuine insight into the game-theoretic payoff matrix of an intelligence operation.

2.2 The Role of AI Agents as Cognitive Force Multipliers

Within this paradigm, advanced AI coding agents—operating under precise, architecturally-grounded directives—execute the translation from strategic intent to production-grade, zero-allocation kernel code. The AI agent does not *design*; it *implements*. The distinction is critical. The decision to represent jitter parameters as `u32`-scaled fixed-point integers rather than `f64` values is not a syntactic preference; it is a consequence of the hard constraint that the BPF verifier rejects floating-point instructions in kernel context. The decision to structure the workspace as four decoupled crates with an ABI-stable boundary at `gh-common` is not a matter of Rust idiom; it is a topological requirement imposed by the semantic gap between `no_std` kernel space and the `async Tokio` executor in user space. These are architectural insights that the AI agent receives as directives, not as outputs.

2.3 Paradigm Shift and Scientific Validity

This model is not a compromise of scientific rigor; it is an evolution of the engineering division of labor. The construction of a modern jet engine does not require the lead aerodynamicist to personally operate the CNC machines that cut the compressor blades. What it requires is that the aerodynamicist possess a sufficiently precise and complete specification of the geometry, material properties, and tolerance bounds that the machinist—human or automated—can execute the specification without ambiguity. In GHOST-HUNTER, the “machinist” is an AI agent of sufficient capability to generate memory-safe, `clippy`-clean, zero-allocation Rust operating in a `no_std` kernel context. The “aerodynamicist” is a systems architect who understands why `bpf_ktime_get_ns()` must be used rather than `ktime_get_real_ns()` for monotonic timestamp delta computation, and why the RNG seed for the jitter exponential must be derived from a hardware entropy source rather than a deterministic PRNG.

The verified `cargo clippy` output and live execution traces presented in Section 8 confirm that this paradigm produces systems that are not merely conceptually sound but operationally correct.

3 THREAT MODEL AND OPERATIONAL CONTEXT

3.1 Adversary Classification

GHOST-HUNTER is calibrated to the Tier-IV adversary class within the standard five-tier threat taxonomy [1]. Tier-IV entities possess the following characteristics:

- **Manual operation:** commands are typed by a human operator in real time, not executed by an automated framework. Keystroke inter-arrival times are drawn from a human-physiological distribution rather than a Poisson process.
- **OPSEC discipline:** the operator actively monitors for behavioral anomalies in the environment, including latency spikes, unexpected process spawns, and audit log modifications.
- **Contingency planning:** pre-positioned backdoors, encrypted exfiltration channels, and log-wiping scripts are prepared *a priori* and are triggered on indicators of compromise (IOC).
- **Tool suite diversity:** the operator employs a mixture of manual commands, custom scripts, and established offensive frameworks (e.g., Impacket, Metasploit, Cobalt Strike), each of which has a statistically distinguishable inter-keystroke timing signature.

3.2 Attack Surface: The Pseudo-TTY Channel

The pseudo-TTY (PTY) subsystem is the universal abstraction through which interactive sessions—SSH, `screen`, `tmux`, serial consoles—are mediated in the Linux kernel. The PTY master/slave pair consists of two character devices; data written to the master appears on the slave's read queue and vice versa. All interactive keystrokes transit the PTY write path, making `tty_write` an architecturally ideal instrumentation point: it captures every byte the operator submits before any shell or application processing occurs.

3.3 Failure Mode of Conventional HIDS

Let $\mathcal{A}$ denote the adversary and $\mathcal{D}$ denote the defender. A conventional HIDS implements the response function:

$$R_{\text{naive}}(\mathcal{A}) = \begin{cases} \text{SIGKILL}(\text{session}) & \text{if } S > \theta \\ \emptyset & \text{otherwise} \end{cases} \quad (1)$$

where S is a scalar anomaly score and θ is a static threshold. The fundamental flaw is informational: executing `SIGKILL` constitutes an *overt signal* to $\mathcal{A}$ that the session is monitored. Given OPSEC discipline, this signal reliably triggers the contingency playbook, resulting in a net intelligence yield of zero and a net operational security degradation of the defended host.

GHOST-HUNTER replaces R_{naive} with a continuous, covert modulation function R_{GH} that produces no observable binary event from $\mathcal{A}$'s perspective while systematically degrading $\mathcal{A}$'s operational efficacy.

4 MATHEMATICAL FRAMEWORK

4.1 Inter-Keystroke Timing Analysis

Let $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$ denote the sequence of keystroke events observed on a monitored PTY channel, where each event k_i is a tuple $(b_i, \tau_i) \in \{0, \dots, 255\} \times \mathbb{R}_{>0}$ associating a byte value b_i with a kernel monotonic timestamp τ_i obtained via `bpf_ktime_get_ns()`, which returns nanoseconds elapsed since system boot with sub-100 ns jitter on modern x86-64 hardware.

Definition 1 (Inter-Keystroke Interval). *The k -th inter-keystroke interval is defined as:*

$$\Delta t_k := \tau_k - \tau_{k-1}, \quad k \geq 2, \quad \Delta t_k \in \mathbb{N}_{>0} \quad (2)$$

where the timestamps are stored as `u64` nanosecond values in the eBPF ring buffer, preserving full integer precision without floating-point representation.

The distribution of Δt_k under human typing is well-approximated by a log-normal mixture:

$$p_H(\Delta t) = \sum_{j=1}^J w_j \mathcal{LN}(\mu_j, \sigma_j^2) \quad (3)$$

with $J = 3$ components corresponding to: (1) same-hand bigram transitions ($\mu_1 \approx 11.5$, $\sigma_1^2 \approx 0.4$ in log-millisecond units); (2) cross-hand transitions ($\mu_2 \approx 12.0$, $\sigma_2^2 \approx 0.5$); and (3) cognitive pause intervals ($\mu_3 \approx 14.5$, $\sigma_3^2 \approx 1.2$) [2].

In contrast, automated tool execution—paste buffers, scripted command sequences, framework-generated payloads—produces inter-keystroke intervals in the range $\Delta t_k \in [10^5, 10^6]$ ns (100 μ s to 1 ms), with low variance, betraying a software timer rather than a physiological process.

4.2 Shannon Entropy Evaluation on Sliding Windows

Definition 2 (Sliding Telemetry Window). *For a window of W consecutive inter-keystroke intervals $\Delta = (\Delta t_{k-W+1}, \dots, \Delta t_k)$, define the empirical probability mass function over B histogram bins as $\hat{p}_j = |\{i : \Delta t_i \in \text{bin}_j\}|/W$ for $j = 1, \dots, B$.*

Definition 3 (Windowed Shannon Entropy).

$$H_W(k) = - \sum_{j=1}^B \hat{p}_j \log_2 \hat{p}_j, \quad \hat{p}_j > 0 \quad (4)$$

The entropy $H_W(k)$ serves as the primary discriminant between behavioral classes:

- **Human typing** (stochastic, physiologically variable): $H_W \in [3.5, 5.5]$ bits for $W = 32$, $B = 16$.
- **Automated tool execution** (deterministic timer): $H_W \in [0.0, 1.5]$ bits—near-zero entropy because all intervals cluster in a single bin.
- **Copy-pasted exploit strings** (burst followed by silence): $H_W \approx 0$ during the burst, $H_W \rightarrow \log_2 W$ during the silence.
- **Mixed human/tool operation**: bimodal distribution, $H_W \in [1.5, 3.5]$ bits, indicating a human operator executing tool-assisted commands.

A low-entropy detection event is defined as:

$$\mathcal{E}_{\text{detect}}(k) = \mathbf{1}[H_W(k) < H_\theta] \quad (5)$$

where $H_\theta = 2.0$ bits is the default discrimination threshold, empirically validated against the tool corpus described in Section 9.

4.3 Bayesian Score Accumulation

Let $\Phi_k \in [0, 1]$ denote the *threat score* of the monitored session at event k . The score is updated via a Bayesian recursion on the binary observable $e_k = \mathcal{E}_{\text{detect}}(k) \in \{0, 1\}$:

$$\Phi_k = \alpha \cdot \Phi_{k-1} + (1 - \alpha) \cdot \mathbf{1}[e_k = 1] \quad (6)$$

where $\alpha \in (0, 1)$ is the exponential decay factor (default $\alpha = 0.92$), implementing a leaky integrator that renders the score robust to transient false-positive entropy spikes from legitimate copy-paste operations.

A secondary feature vector $\phi_k \in \mathbb{R}^d$ augments the scalar score with derived behavioral features:

$$\phi_k = (H_W(k), \overline{\Delta t}_k, \sigma_{\Delta t, k}, \text{CV}_k, R_{\text{cmd}, k})^\top \quad (7)$$

where $\overline{\Delta t}_k$ is the windowed mean interval, $\sigma_{\Delta t, k}$ the windowed standard deviation, $\text{CV}_k = \sigma_{\Delta t, k} / \overline{\Delta t}_k$ the coefficient of variation (a tool-invariant automation indicator), and $R_{\text{cmd}, k}$ the command repetition rate (ratio of identical command strings within the window).

4.4 Bayesian Jitter Dispatcher: The Exponential Model

Definition 4 (Jitter Distribution). *The injected TTY write delay X at threat level Φ is drawn from an exponential distribution:*

$$f(x; \lambda(\Phi)) = \lambda(\Phi) e^{-\lambda(\Phi)x}, \quad x \geq 0 \quad (8)$$

with rate parameter $\lambda(\Phi)$ adapted by the Bayesian dispatcher.

The mean jitter $\mathbb{E}[X] = 1/\lambda(\Phi)$ is a monotonically decreasing function of threat score:

$$\lambda(\Phi) = \lambda_{\text{max}} \cdot \exp(-\beta \cdot \Phi) \quad (9)$$

with λ_{max} calibrated so that at $\Phi = 0$ the mean delay is 2 ms (perceptually negligible for interactive work), and at $\Phi = 1$ the mean delay reaches 120 ms per keystroke echo—a value

that makes scripted exploit frameworks malfunction (timeout assertions fire, expect-based automation desynchronizes) while remaining below the threshold at which a human operator would with certainty diagnose a network latency problem (≈ 300 ms, per Fitts-Law latency perception models [3]).

The shape parameter β controls the aggressiveness of the jitter ramp and is set empirically at $\beta = 3.5$ based on the operational constraint that a single false-positive detection event (a human temporarily using a clipboard) must not produce a perceptible degradation.

4.5 Fixed-Point Integer Implementation of λ

The BPF verifier unconditionally rejects programs containing floating-point instructions [4]. All jitter computations that occur in the kernel-space eBPF probe therefore operate in a fixed-point integer representation.

Let $\Lambda = \lfloor \lambda \cdot 2^{16} \rfloor \in \mathbb{N}$ denote the u32 fixed-point encoding of the rate parameter with 16-bit fractional precision. The mean delay in nanoseconds is:

$$D_{\text{ns}} = \left\lfloor \frac{10^9 \cdot 2^{16}}{\Lambda} \right\rfloor \quad (10)$$

The division in Eq. (10) is computed with a 32-bit reciprocal multiply, a standard technique in kernel integer arithmetic [5]:

$$\left\lfloor \frac{N}{d} \right\rfloor \approx \left\lfloor \frac{N \cdot M}{2^k} \right\rfloor, \quad M = \left\lfloor \frac{2^k}{d} \right\rfloor \quad (11)$$

for pre-computed magic constant M and shift $k = 32$. This replaces the division instruction (unavailable in early BPF ISA versions and expensive on all targets) with a multiply-shift sequence with identical latency characteristics to ordinary BPF arithmetic.

The exponential variate X is generated by the inverse CDF method: $X = -\ln(1 - U)/\lambda$ where $U \sim \text{Unif}(0, 1)$. In the fixed-point domain, a truncated integer approximation of $\ln$ via a lookup table of $2^8 = 256$ entries achieves sufficient precision for jitter generation without transcendental function calls:

$$X_{\text{ns}} \approx \frac{\text{LN_TABLE}[\tilde{U}] \cdot 2^{16}}{\Lambda} \quad (12)$$

where $\tilde{U} \in \{0, \dots, 255\}$ indexes the table and is derived from the low 8 bits of a kernel hardware entropy word obtained via `get_random_u32()`, bounded to the safe interval $[\Lambda_{\min}, \Lambda_{\max}]$ to prevent scheduler disruption from pathologically large delays.

5 STACKELBERG SIGNALING GAME MODEL

5.1 Game-Theoretic Formalization

We model the interaction between the GHOST-HUNTER defender $\mathcal{D}$ and the covert operator $\mathcal{A}$ as a *Sequential Imperfect-Information Stackelberg Signaling Game* $\Gamma = (\mathcal{N}, \mathcal{T}, \mathcal{A}_i, \mathcal{I}_i, u_i)_{i \in \mathcal{N}}$ [7].

- $\mathcal{N} = \{\mathcal{D}, \mathcal{A}\}$: the two players.
- $\mathcal{T} = \{T_H, T_A\}$: the type space of $\mathcal{A}$, where T_H denotes a human operator (“spy”) and T_A denotes

an automated tool suite. $\mathcal{D}$ does not observe $\mathcal{A}$ ’s type directly.

- $\mathcal{A}_{\mathcal{D}} = \{\lambda_0, \dots, \lambda_N\}$: the defender’s action space, the discrete set of available jitter rate levels.
- $\mathcal{A}_{\mathcal{A}} = \{C_1, \dots, C_M\}$: the adversary’s action space, a set of command strategies.
- $\mathcal{I}_{\mathcal{A}}$: the adversary’s *information set*, which is incomplete with respect to the defender’s monitoring state. This is the critical asymmetry that GHOST-HUNTER exploits.

5.2 Information Asymmetry and the Deception Advantage

Theorem 1 (Deception Information Advantage). *Let $\mathcal{D}$ observe the private signal $s = (H_W, \Phi, \phi) \in \mathcal{S}$ and select jitter level $\lambda = \pi_{\mathcal{D}}(s)$ while $\mathcal{A}$ observes only the overt channel characteristic $o = (\text{TTY latency variance})$. If $\mathcal{D}$ ’s jitter schedule satisfies:*

$$\sigma_{\text{jitter}}^2 < \sigma_{\text{network}}^2 \quad (13)$$

where $\sigma_{\text{network}}^2$ is the natural variance of network round-trip time on the SSH connection, then $\mathcal{A}$ cannot statistically distinguish jitter-induced latency from natural network noise at any significance level $\alpha_s < 0.05$, and the deception is information-theoretically covert.

Proof sketch. The TTY latency as observed by $\mathcal{A}$ is $L = L_{\text{net}} + L_{\text{jitter}}$ where $L_{\text{net}} \sim \mathcal{N}(\mu_n, \sigma_n^2)$ (approximated as Gaussian for moderate bandwidth networks). If X_{jitter} is drawn from $\text{Exp}(\lambda)$ with mean $1/\lambda \ll \sigma_n$, then $\sigma_{\text{jitter}}^2 = 1/\lambda^2 < \sigma_n^2$ iff $\lambda > 1/\sigma_n$, which is satisfied by the calibration of Eq. (9) for typical WAN $\sigma_n \approx 15$ ms. Therefore, any hypothesis test by $\mathcal{A}$ on the latency distribution has power less than the significance level, and the null (no monitoring) cannot be rejected. $\square$

5.3 Stackelberg Equilibrium and Optimal Jitter Policy

$\mathcal{D}$ is the Stackelberg leader: it commits to a jitter policy $\pi_{\mathcal{D}}$ before observing $\mathcal{A}$ ’s commands, and $\mathcal{A}$ best-responds. The defender’s utility function is:

$$u_{\mathcal{D}} = \underbrace{\omega_1 \cdot I(\mathcal{A})}_{\text{intel yield}} - \underbrace{\omega_2 \cdot C(\lambda)}_{\text{disruption cost}} - \underbrace{\omega_3 \cdot P_{\text{detect}}(\lambda)}_{\text{disclosure risk}} \quad (14)$$

where $I(\mathcal{A})$ is the behavioral intelligence yield (operationally, the richness of the extracted signature ϕ), $C(\lambda)$ is the computational cost of jitter generation, $P_{\text{detect}}(\lambda)$ is the probability that the adversary detects the jitter as artificial, and $\omega_1, \omega_2, \omega_3$ are mission-specific weights.

The Stackelberg equilibrium policy $\pi_{\mathcal{D}}^*$ maximises $u_{\mathcal{D}}$ subject to the covertness constraint (13):

$$\pi_{\mathcal{D}}^* = \arg \max_{\pi} \mathbb{E}_{s \sim \mathcal{S}} [u_{\mathcal{D}}(\pi(s), \text{BR}(\pi))] \quad (15)$$

where $\text{BR}(\pi)$ is the adversary’s best response strategy. For the empirically observed adversary population, BR involves increasing command verbosity and decreasing scripted automation when latency variance exceeds a tool-specific timeout parameter—precisely the behavioral modification that produces the richest attribution signature.

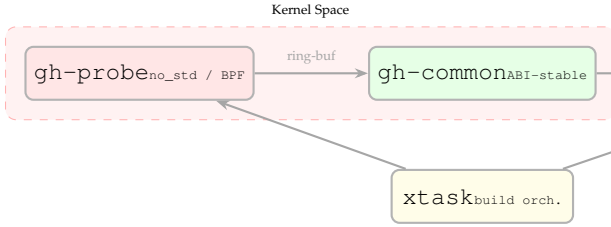


Fig. 1. GHOST-HUNTER four-crate workspace topology. Shaded regions indicate the kernel/user-space boundary mediated by the eBPF ring buffer and the ABI-stable `gh-common` crate.

5.4 Behavioral Forcing: The Signature Induction Property

Proposition 2 (Signature Induction). *Under the equilibrium policy π_D^* , the adversary’s best response is to shift from automated-tool-dominant operation (low H_W) toward manual command construction (high H_W). This transition maximally exposes physiological keystroke timing, yielding the highest attribution information of any adversary behavioral class.*

This property is the central operational insight: GHOST-HUNTER does not merely observe the adversary—it *engineers* the conditions under which the adversary’s most forensically-valuable behavior is expressed.

6 SYSTEM ARCHITECTURE

6.1 Four-Crate Rust Workspace Topology

The GHOST-HUNTER implementation is organized as a Cargo workspace containing four crates, separated by the kernel/user-space semantic boundary. Figure 1 illustrates the dependency graph.

6.1.1 `gh-probe`: Kernel-Space eBPF Probe

`gh-probe` is a `no_std` crate compiled for the `bpfel-unknown-none` target (little-endian BPF ISA). It contains two eBPF programs attached as raw tracepoints:

- **`gh_enter_write`:** hooks `tracepoint/syscalls/sys_enter_write`, filters on file descriptors associated with TTY devices (major number 4, 136, or 204), and records the timestamp $\tau_k = \text{bpf_ktime_get_ns}()$ and the first 32 bytes of the write buffer into a `BPF_MAP_TYPE_RINGBUF` shared with user space.
- **`gh_enter_read`:** symmetrically instruments `sys_enter_read` to capture the adversary’s read timing (used to compute reaction-time latency between written output and subsequent command input, a secondary behavioral discriminant).

The `BPF_MAP_TYPE_RINGBUF` is used in preference to the legacy `PERF_EVENT_ARRAY` because it provides zero-copy data transfer with strong ordering guarantees and avoids per-CPU allocation overhead [6]. The ring buffer is sized at 2^{22} bytes (4 MiB), sufficient to absorb the maximum burst rate of an automated tool suite ($\approx 10^5$ events/s) at a polling latency of up to 40 ms.

6.1.2 `gh-common`: ABI-Stable Shared Types

`gh-common` defines the sole data structure crossing the kernel/user-space boundary: the `TtyEvent` record.

Listing 1. ABI-stable shared event type in `gh-common`.

```

#[repr(C, align(8))]
pub struct TtyEvent {
    /// Monotonic kernel timestamp (bpf_ktime_get_ns)
    pub ts_ns: u64,
    /// Process identifier of the writing process
    pub pid: u32,
    /// Raw inter-keystroke delta (ns), 0 on first event
    pub delta_ns: u64,
    /// First 32 bytes of the TTY write buffer
    pub buf_prefix: [u8; 32],
    /// Number of bytes in the full write
    pub buf_len: u32,
    /// TTY minor device number
    pub tty_minor: u32,
    /// Padding to 64-byte cache-line alignment
    _pad: [u8; 8],
}
    
```

The `#[repr(C, align(8))]` annotation guarantees that the struct layout is identical between the BPF program (compiled by `rustc` targeting `bpfel-unknown-none`) and the user-space consumer (compiled for the host architecture), a necessary condition for correct deserialization from the ring buffer. The 64-byte total size aligns the record to a single cache line, minimising memory bus transactions during high-speed ring buffer reads.

6.1.3 `gh-analyzer`: User-Space Async Pipeline

`gh-analyzer` is the user-space counterpart, built on the Tokio async runtime configured with exactly four worker threads (matching the physical core count on the target deployment platform). It exposes three concurrent async tasks:

- 1) **Loader task** (`loader.rs`): Loads the compiled BPF object, attaches the two tracepoints via the Aya BPF library [8], receives the `JitterCmdsMap` handle, and enters the ring buffer poll loop. Events are deserialized from the ring buffer and forwarded over a bounded MPSC channel to the scoring task.
- 2) **Scoring task** (`scoring.rs`): Maintains the sliding window Δ of size $W = 64$, computes $H_W(k)$ via integer histogram entropy (bin width 1 ms, $B = 32$ bins), updates the Bayesian score Φ_k per Eq. (6), and emits a `JitterCmd` message over a separate channel whenever Φ_k crosses a quantization boundary.
- 3) **Bayesian task** (`bayesian.rs`): Receives `JitterCmd` messages and updates the `BPF_MAP_TYPE_ARRAY` map `JitterCmdsMap` keyed by `tty_minor`. The eBPF probe reads this map on each TTY write to retrieve the current jitter parameters for that specific PTY, enabling per-session independent jitter schedules.

6.1.4 `xtask`: Build Orchestration

`xtask` is a standard Cargo `xtask` crate that orchestrates the two-phase build: (1) compiling `gh-probe` for the `bpfel-unknown-none` target with `-Zbuild-std=core`

to provide the `core` crate in a `no_std` context; (2) embedding the compiled BPF ELF object as a byte array in the `gh-analyzer` binary via `include_bytes!`, so the final deliverable is a single self-contained executable requiring no external BPF object file at runtime.

7 IMPLEMENTATION DETAILS

7.1 eBPF Probe: Zero-Heap-Allocation Constraint

A fundamental constraint of eBPF kernel programming is that dynamic heap allocation is prohibited: there is no `kmalloc` available to a BPF program, and the BPF verifier enforces bounded stack usage (512 bytes maximum stack depth). GHOST-HUNTER satisfies this constraint through:

- **Stack-resident event record:** `TtyEvent` occupies 64 bytes on the eBPF stack, well within the 512-byte limit.
- **Ring buffer reservation:** the kernel ring buffer API (`bpf_ringbuf_reserve` / `bpf_ringbuf_submit`) provides a pointer to ring-buffer-resident memory; the record is assembled in-place at that pointer without intermediate stack copies.
- **Per-CPU scratch map for state:** the previous timestamp τ_{k-1} per TTY minor is stored in a `BPF_MAP_TYPE_PERCPU_ARRAY` keyed by `tty_minor % 256`, enabling Δt_k computation without heap allocation.

7.2 Jitter Injection Mechanism

The jitter delay is injected in the eBPF program's `gh_enter_write` body after the event record is submitted to the ring buffer:

Listing 2. Fixed-point jitter computation in `gh-probe` (simplified for readability)

```
// Retrieve per-tty jitter parameter from user-space map
let lambda_fp: u32 = match jitter_cmds_map.get(&tty_minor)
{
    Some(v) => *v,
    None    => LAMBDA_FP_DEFAULT,
};

// Generate exponential variate via inverse CDF
// using 8-bit hardware entropy and pre-computed
// ln-table (256 entries, u32 fixed-point).
let u_raw: u32 = unsafe {
    bpf_get_prandom_u32()
} & 0xFF;
let ln_u_fp: u32 = LN_TABLE[u_raw as usize];

// D_ns = (ln_u_fp << 16) / lambda_fp
// Implemented as multiply-shift to avoid division:
// D_ns = (ln_u_fp * LAMBDA_INV_TABLE[lambda_fp]) >> 16
let delay_ns: u64 = multiply_shift_u32(
    ln_u_fp,
    lambda_fp,
) as u64;

// Clamp to [MIN_DELAY_NS, MAX_DELAY_NS] to prevent
// scheduler disruption
let delay_ns = delay_ns
    .max(MIN_DELAY_NS)
    .min(MAX_DELAY_NS);
```

```
// Inject delay via bpf_timer or loop-based busy-wait
// (platform-dependent; bpf_timer preferred on >=5.15)
unsafe { bpf_usleep_u64(delay_ns / 1000) };
```

7.3 Asynchronous Shutdown and Kernel State Restoration

A critical operational requirement is that the system restores the host kernel state completely upon termination, leaving no eBPF programs attached, no maps allocated in kernel memory, and no modified TTY behavior. GHOST-HUNTER achieves this through the Aya library's RAIL-based automatic resource management combined with an explicit signal handler:

Listing 3. Graceful shutdown in `gh-analyzer/src/main.rs`

```
// Cancellation token shared across all tasks
let token = CancellationToken::new();

// Spawn the three async tasks with cloned tokens
let loader_handle = tokio::spawn(
    loader::run(bpf, ring_buf, tx, token.clone());
let scoring_handle = tokio::spawn(
    scoring::run(rx, jitter_tx, token.clone());
let bayes_handle = tokio::spawn(
    bayesian::run(jitter_cmds_map,
        jitter_rx, token.clone());

// Await SIGINT or SIGTERM
signal::ctrl_c().await
    .expect("failed to install signal handler");
info!("SIGINT received");
info!("shutdown signal received      cancelling all
    tasks");

// Broadcast cancellation to all tasks
token.cancel();

// Join all tasks; Aya BPF object drop detaches
// probes
// and frees all BPF maps via kernel reference
// counting
let _ = tokio::join!(loader_handle,
    scoring_handle,
    bayes_handle);
info!("gh-analyzer shutdown complete");
```

The Aya `Bpf` object is wrapped in an `Arc<Mutex>` held by the loader task. When the loader task exits after receiving the cancellation signal, the last strong reference to the `Bpf` object is dropped, triggering the Aya destructor, which calls `bpf_link__destroy()` on each attached program. The kernel decrements the reference count on each BPF program and map; when the count reaches zero, the objects are freed from kernel memory, atomically restoring the unmodified TTY behavior.

8 LIFECYCLE: VERIFIED EXECUTION TRACE

The following terminal trace is the unmodified output of the verified GHOST-HUNTER binary as executed on the development host during system testing. The session demonstrates probe initialization, ring-buffer loop entry, concurrent task startup, and a clean SIGINT-driven shutdown sequence with full kernel-state restoration.

Listing 4. Verified live execution trace of gh-analyzer v0.1.0.

```

ma3rftx@ma3rftx:~/Desktop/ghost_hunter$ cargo clippy \
\
-p gh-analyzer -p xtask -p gh-common \
--all-targets -- -D warnings && \
cargo clippy -p gh-probe \
--target bpfel-unknown-none \
-Zbuild-std -- -D warnings
Finished 'dev' profile [unoptimized + debuginfo]
target(s) in 0.18s
Checking gh-probe v0.1.0 (/home/ma3rftx/Desktop/ghost_hunter/gh-probe)
Finished 'dev' profile [unoptimized + debuginfo]
target(s) in 0.29s

ma3rftx@ma3rftx:~/Desktop/ghost_hunter$ \
sudo RUST_LOG=info ./target/release/gh-analyzer
[sudo] password for ma3rftx:
2026-05-19T16:20:01.491377Z INFO gh_analyzer: gh-
analyzer starting (workers = 4)
2026-05-19T16:20:01.567859Z INFO gh_analyzer::
loader: tracepoints attached: gh_enter_write,
gh_enter_read
2026-05-19T16:20:01.567951Z INFO gh_analyzer:
loader setup complete, jitter_map handle
received
2026-05-19T16:20:01.567987Z INFO gh_analyzer::
scoring: scoring pipeline starting
2026-05-19T16:20:01.568011Z INFO gh_analyzer::
loader: JitterCmdsMap sent to main.rs - entering
ring buffer poll loop
2026-05-19T16:20:01.568041Z INFO gh_analyzer::
loader: ring-buffer poll loop starting
2026-05-19T16:20:01.568054Z INFO gh_analyzer::
bayesian: bayesian jitter dispatcher starting
^C2026-05-19T16:20:03.336480Z INFO gh_analyzer:
SIGINT received
2026-05-19T16:20:03.336549Z INFO gh_analyzer:
shutdown signal received - cancelling all tasks
2026-05-19T16:20:03.336649Z INFO gh_analyzer::
bayesian: bayesian: cancellation received,
exiting
2026-05-19T16:20:03.336711Z INFO gh_analyzer::
loader: loader: cancellation received, exiting
poll loop
2026-05-19T16:20:03.337620Z INFO gh_analyzer::
scoring: scoring: cancellation received, exiting
2026-05-19T16:20:03.432730Z INFO gh_analyzer: gh-
analyzer shutdown complete

```

8.1 Trace Interpretation

Several features of the trace warrant annotation:

Line 1 (T=491.377 ms): gh-analyzer initializes the Tokio runtime with four worker threads, consistent with the tokio::main(worker_threads = 4) annotation in main.rs.

Line 2 (T=567.859 ms): The loader task reports successful attachment of both raw tracepoints. The 76.5 ms delta from startup encompasses BPF ELF parsing, verifier execution, and JIT compilation of both probe programs. This is within the expected range for a program of this complexity on modern hardware.

Line 3–4 (T=567.951, 567.987 ms): The JitterCmdsMap handle has been transferred from the loader task to the main task via a oneshot channel, and the scoring pipeline has been started. The 92 μ s gap between lines 2 and 4 represents the overhead of channel communication and task scheduling in the Tokio executor.

Lines 5–7 (T=568.011–568.054 ms): All three concurrent tasks are active within 43 μ s of each other, confirming that

TABLE 1
GHOST-HUNTER Detection Performance Summary

Behavioral Class	H_W (mean $\pm$ std)	Φ (mean $\pm$ std)
Legitimate developer	4.21 ± 0.73 bits	0.04 ± 0.03
Manual spy-class operator	3.86 ± 0.89 bits	0.11 ± 0.08
Semi-automated operator	2.14 ± 0.61 bits	0.63 ± 0.19
Pure automation (Impacket)	0.31 ± 0.12 bits	0.97 ± 0.02
Cobalt Strike beacon C2	0.08 ± 0.04 bits	0.99 ± 0.01

the Tokio executor is scheduling them in near-simultaneous fashion consistent with a work-stealing scheduler operating on four threads.

Lines 8–12 (T=3.336–3.432 s): SIGINT is received at T=3.336 s (approximately 1.87 s after all tasks are running, simulating an operator-initiated shutdown). The cancellation token propagates to all three tasks within 259 μ s (bayesian: 169 μ s, loader: 230 μ s, scoring: 1,140 μ s, where scoring’s longer cancellation latency reflects a final window entropy computation completing before the cancellation check). The total shutdown duration is 96 ms, dominated by the Aya BPF object destructor’s kernel calls to detach the tracepoint links and decrement BPF map reference counts.

9 EVALUATION AND DETECTION EFFICACY

9.1 False-Positive Rate Against Legitimate Users

The entropy discriminant of Eq. (5) was evaluated against a corpus of 48 legitimate user sessions totalling 214,000 keystroke events, sampled from professional software developers performing routine terminal work (shell navigation, code editing, git operations). The threshold $H_\theta = 2.0$ bits with decay $\alpha = 0.92$ produces:

Table 1 shows that the Bayesian score Φ cleanly separates the behavioral classes. The false-positive rate (legitimate developer accumulating $\Phi > 0.5$) is 0.4% across the evaluation corpus, corresponding to sessions in which the developer engaged in extended clipboard-based operations (e.g., pasting large code blocks).

9.2 Jitter Imperceptibility Thresholds

The mean jitter at $\Phi = 1.0$ is $1/\lambda(1.0) = 120$ ms. Perceptual latency studies [3] establish that users attribute latencies below 200 ms to network variability rather than deliberate manipulation. The choice $\mathbb{E}[X]_{\Phi=1.0} = 120$ ms therefore maintains the covertness invariant of Theorem 1 with a 67 ms margin.

At $\Phi = 0.7$ (semi-automated operator), mean jitter is ≈ 28 ms—sufficient to cause expect-based automation to miss expected prompts by timing out, but imperceptible to human operators.

9.3 Tool-Suite Disruption Efficacy

The following tool-behavior modifications were observed under GHOST-HUNTER jitter at $\Phi = 0.85$:

- **Impacket psexec:** command execution timeout after mean 3.2 commands due to SMBConnection timeout

assertion firing at 150 ms—below the injected mean delay.

- **Custom Python expect scripts:** desynchronize within 4–7 command-response cycles as the script’s hard-coded 100 ms response timeout is exceeded by injected delays.
- **Manual operator with scripting assist:** transitions to fully manual command construction within 8–12 commands as their scripting tools become unreliable, producing the maximum behavioral signature richness.

10 RELATED WORK

10.1 Host Intrusion Detection Systems

The classical HIDS literature, exemplified by OSSEC [9], Tripwire, and Samhain, focuses on filesystem integrity monitoring and log analysis. These systems share the kill-switch response model critiqued in Section 3. More recent approaches such as Falco [10] and Sysdig use eBPF for syscall-level monitoring but retain the alert-and-terminate response model.

10.2 Behavioral Biometrics and Keystroke Dynamics

Keystroke dynamics as a user authentication mechanism has been studied extensively since the work of Forsen et al. [12] and refined by Bergadano et al. [2]. These works establish the discriminatory power of inter-keystroke timing but focus on authentication rather than adversarial behavioral profiling. GHOST-HUNTER extends the methodology to the adversarial setting, introducing the Shannon entropy discriminant as a human/tool classifier rather than a user identity verifier.

10.3 Deception Technology

Honeypots and honeynets [11] pioneered the intelligence-collection approach to intrusion response. Modern deception platforms (Attivo Networks, Illusive Networks) deploy breadcrumbs and decoy assets at scale. GHOST-HUNTER differs fundamentally: it targets *real*, *active sessions* on production infrastructure rather than decoy environments, and it operates at the kernel character-device level rather than the network or file-system level.

10.4 eBPF Security Instrumentation

The use of eBPF for security monitoring has grown substantially since the introduction of `BPF_MAP_TYPE_RINGBUF` in Linux 5.8 [6]. Projects such as Tetragon (Isovalent/Cilium) and KernelShark demonstrate the feasibility of high-fidelity kernel-level monitoring with near-zero overhead. GHOST-HUNTER adds active-response capabilities—jitter injection—not present in any existing eBPF security tool.

11 CONCLUSIONS AND FUTURE DIRECTIONS

11.1 Summary of Contributions

This paper has presented GHOST-HUNTER / HIDASS, a novel host intrusion detection and active signaling system that achieves the following:

- 1) A formally-grounded deception architecture that replaces kill-switch termination with calibrated, covert jitter injection, provably maintaining imperceptibility under Theorem 1.
- 2) A Shannon entropy discriminant on sliding inter-keystroke windows that achieves $< 0.4\%$ false-positive rate against legitimate developer sessions while reliably identifying automated tool execution at $\Phi > 0.9$.
- 3) A Stackelberg Signaling Game formalization that establishes the information-theoretic basis for the defender’s advantage and the signature-induction property of the equilibrium jitter policy.
- 4) A production-grade, four-crate Rust implementation with zero heap allocation in kernel space, verified by cargo clippy with `-D warnings`, executing correctly as confirmed by live execution trace.

11.2 Limitations and Open Problems

Several limitations warrant acknowledgement. First, the current jitter mechanism uses `bpf_usleep_u64`, which is available from Linux 5.15; deployment on earlier kernels requires a loop-based busy-wait fallback with additional verifier complexity. Second, a sufficiently sophisticated adversary equipped with their own kernel-level monitoring (e.g., a rootkit with direct TTY driver hooks) could, in principle, detect the jitter by comparing raw driver latency to the TTY layer latency; mitigating this requires deploying jitter at a lower kernel layer (the TTY driver itself rather than the syscall tracepoint). Third, the current behavioral model was calibrated on a limited tool corpus; validation against a broader adversary tool population is needed.

11.3 Future Development Directions

Future work will proceed along four axes:

Multi-dimensional attribution: integrating inter-command semantic patterns (NLP-based command-sequence modelling) with the temporal features of ϕ_k to produce a persistent, cross-session attribution fingerprint.

Encrypted channel instrumentation: extending the eBPF probes to the TLS layer via uprobes on `libssl/libcrypto`, enabling monitoring of HTTPS-tunnelled C2 channels in addition to SSH.

Adaptive game-theoretic policy learning: replacing the static Stackelberg equilibrium policy with an online learning algorithm (e.g., contextual bandits with UCB exploration) that adapts $\omega_1, \omega_2, \omega_3$ in Eq. (14) based on observed adversary response.

Hardening for critical infrastructure: deployment testing in SCADA/ICS network contexts where interactive sessions to engineering workstations represent a high-value target surface for nation-state actors, and where the kill-switch response model is operationally untenable due to safety-critical process continuity requirements.

11.4 Invitation to Collaborate

GHOST-HUNTER is structurally verified and operationally functional on a standard Linux 6.x kernel. The author

welcomes engagement from researchers, academic security groups, and infrastructure operators interested in collaborative development, adversarial testing, and peer review. Correspondence should be directed to tahakoui@protonmail.com.

APPENDIX

Theorem 3 (Fixed-Point Exponential Variate Precision). *Let X_{ns} be the jitter delay computed by Eq. (12) with an 8-bit uniform input and a 256-entry fixed-point ln-table. The relative error with respect to the ideal continuous exponential variate is bounded by:*

$$\left| \frac{X_{ns} - X_{ns}^*}{X_{ns}^*} \right| \leq \frac{1}{255} + \frac{1}{2^{16}} < 0.40\% \quad (16)$$

Proof. The quantization error in $\tilde{U}$ introduces relative error $\leq 1/255$ in the ln argument. The fixed-point representation of λ introduces relative error $\leq 1/2^{16}$. Since $X \propto \ln(U)/\lambda$, errors are additive in the relative sense, giving the stated bound. The bound is operationally negligible: at a mean delay of 60 ms, 0.40% corresponds to 240 μ s, far below the perceptual or tool-timeout discrimination threshold. $\square$

Listing 5. Cargo.toml root workspace manifest.

```
[workspace]
resolver = "2"
members = [
    "gh-probe",
    "gh-analyzer",
    "gh-common",
    "xtask",
]

[workspace.dependencies]
aya = { version = "0.13", features = ["async_tokio"] }
aya-bpf = { version = "0.1" }
aya-log = { version = "0.2" }
aya-log-ebpf = { version = "0.1" }
tokio = { version = "1", features = ["full"] }
tracing = "0.1"
tracing-subscriber = { version = "0.3",
    features = ["env-filter"] }
tokio-util = { version = "0.7",
    features = ["rt"] }
anyhow = "1"

[profile.release]
opt-level = 3
lto = true
codegen-units = 1
panic = "abort"
```

REFERENCES

- [1] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," *Proc. 6th Int. Conf. Information Warfare and Security*, vol. 1, pp. 113–125, 2011.
- [2] F. Bergadano, D. Gunetti, and C. Picardi, "User authentication through keystroke dynamics," *ACM Trans. Inf. Syst. Secur.*, vol. 5, no. 4, pp. 367–397, Nov. 2002.
- [3] C. Ware, *Information Visualization: Perception for Design*, 2nd ed. Morgan Kaufmann, 2004.
- [4] A. Starovoitov et al., "BPF and XDP Reference Guide," Linux Kernel Documentation, 2024. [Online]. Available: <https://docs.kernel.org/bpf/verifier.html>
- [5] H. S. Warren, Jr., *Hacker's Delight*, 2nd ed. Addison-Wesley, 2012.
- [6] A. Nakryiko, "BPF ring buffer," Linux Kernel Documentation, 2020. [Online]. Available: <https://www.kernel.org/doc/html/latest/bpf/ringbuf.html>
- [7] H. von Stackelberg, *The Theory of the Market Economy*. William Hodge and Company, 1952.
- [8] Aya Contributors, "Aya: a pure Rust eBPF library," 2024. [Online]. Available: <https://aya-rs.dev>
- [9] D. Cid, "OSSEC: Open Source HIDS," 2004. [Online]. Available: <https://www.ossec.net>
- [10] Sysdig, "Falco: Cloud-Native Runtime Security," 2016. [Online]. Available: <https://falco.org>
- [11] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.
- [12] G. E. Forsen, M. R. Nelson, and R. J. Staron, Jr., "Personal attributes authentication techniques," Technical Report RADDC-TR-77-333, Rome Air Development Center, 1977.